CloudBench — Master Project Specification

1. Project Identity

CloudBench — Production-Grade Cloud-Native Software Platform

CloudBench is an engineering-focused portfolio project. The product itself should remain modest; the primary portfolio value comes from demonstrating how a real software system is designed, implemented, tested, deployed, monitored, secured, documented, and maintained.

The project should demonstrate:

- Full-stack software engineering
- API and database design
- Automated testing
- Docker/containerization
- GitHub Actions CI/CD
- Azure deployment
- Infrastructure-as-Code
- Observability
- Security
- API documentation
- Engineering documentation
- Disciplined Git workflow
- Production-oriented architectural thinking

This is a **no-AI/ML application** unless a future requirement explicitly introduces one.

---

2. Project Goal

Build and publicly deploy a small but real software product that demonstrates the ability to take a system from:

```
Idea → Requirements → Architecture → Implementation → Testing
→ Containerization → CI/CD → Infrastructure-as-Code
→ Cloud Deployment → Observability → Documentation
```

The final project should be suitable for:

- A public GitHub repository
- A live public deployment
- Portfolio demonstrations
- Technical interviews
- Resume/project discussions

The product should not become an unnecessarily large SaaS platform.

The engineering surrounding the product is more important than the number of features.

---

3. Technology Stack

Frontend

The frontend is explicitly standardized on:

- **Next.js 15**
- **App Router**
- **React 19**
- **TypeScript**
- **Tailwind CSS v4**
- **Framer Motion**
- **shadcn/ui**
- **Radix UI Primitives**
- **Sonner** for toast notifications

The frontend should demonstrate:

- Next.js App Router architecture
- React component design
- TypeScript
- API consumption
- Server Components vs Client Components where appropriate
- Loading, error, empty, and success states
- Forms and user interaction
- Responsive UI
- Accessible UI
- Meaningful animation
- Toast notifications

Do not let frontend styling become more important than the engineering objectives.

Frontend principles

Use Server Components where they make sense and Client Components only where interactivity requires them.

Use Framer Motion when animation improves feedback, transitions, hierarchy, or perceived responsiveness.

Use Sonner for appropriate transient feedback such as successful actions and recoverable errors. Do not use toasts for information that should permanently exist on the page.

Use shadcn/ui and Radix UI to establish accessible, reusable UI primitives rather than creating every component from scratch.

Backend

The backend is explicitly standardized on:

- **Python**
- **FastAPI**

FastAPI provides the main application API.

The backend should demonstrate:

- API routing
- Request/response schemas
- Validation
- Business-logic separation
- Database access
- Error handling
- Authentication/authorization if required
- Logging
- Testing
- Configuration management
- API documentation

Potential supporting technologies include:

- **Pydantic** for schemas/validation
- **SQLAlchemy** for PostgreSQL access

- **Alembic** for database migrations
- **pytest** for testing
- **httpx** for API/integration testing

These are proposed defaults, not automatic requirements. Each significant dependency must be justified.

For every major technology ask:

1. What problem does it solve?
2. Why do we need it?
3. What happens if we do not use it?
4. What complexity does it introduce?
5. Is the project large enough to justify it?

---

4. Product Scope

The product/domain should be:

- Small
- Understandable
- Useful
- Real enough to justify a database and API
- Suitable for a Next.js frontend
- Suitable for automated testing
- Suitable for public deployment

The product itself should be deliberately modest.

The portfolio value should primarily come from:

- Architecture
- API design
- Database design
- Testing
- CI/CD
- Docker
- Azure
- Infrastructure-as-Code
- Observability
- Security
- Documentation

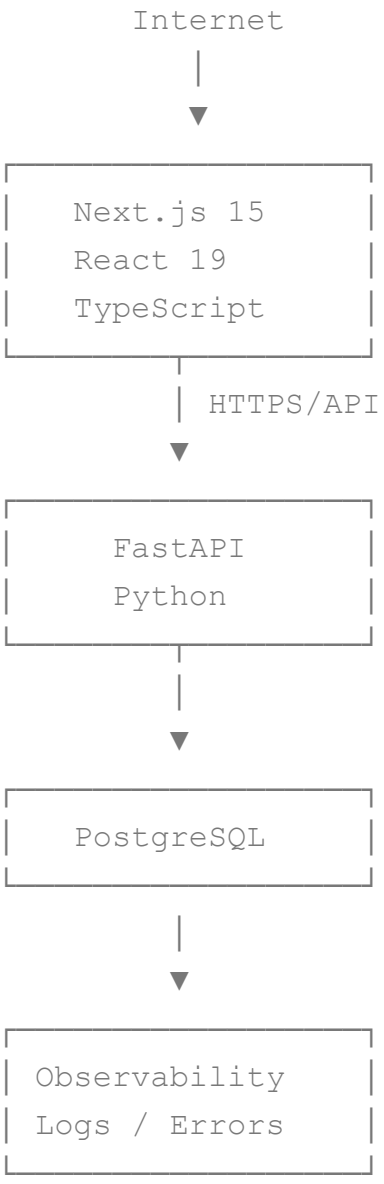
- Git discipline

Do not turn CloudBench into a huge SaaS platform.

---

5. Initial Architecture

Prefer a modular full-stack application:



Automation:

```
Developer
    ↓
GitHub
    ↓
GitHub Actions
```

```
    ├── lint
    ├── typecheck
    ├── test
    ├── build
    └── deploy
            ↓
         Azure
```

The architecture may evolve when actual requirements justify it.

---

6. Architectural Philosophy

Start with the simplest architecture that satisfies the requirements.

Prefer a **modular monolith** initially.

Do not add infrastructure merely because it looks impressive.

Do not automatically introduce:

- Redis
- Kafka
- RabbitMQ
- Kubernetes
- Microservices
- Event-driven architecture
- Service meshes
- Database sharding
- Complex caching
- Distributed systems

unless a concrete requirement justifies them.

A deliberate decision not to use a technology is itself a valid engineering decision.

Before introducing major infrastructure, answer:

1. What problem does it solve?
2. What happens if we do not use it?
3. What complexity does it introduce?
4. What operational cost does it introduce?
5. What failure modes does it introduce?

6. Does current scale justify it?
7. Does it meaningfully improve the project?

---

7. Backend Architecture

Prefer clear separation such as:

```
FastAPI
├── API/routes
├── Schemas/validation
├── Domain/business logic
├── Data-access layer
├── Configuration
├── Infrastructure integrations
└── Tests
```

Do not put all business logic inside route handlers.

The exact structure should be determined after requirements are understood.

---

8. Database

Use **PostgreSQL** as the real persistent database.

Demonstrate:

- Tables
- Relationships
- Primary keys
- Foreign keys
- Constraints
- Appropriate indexes
- Transactions where required
- Migrations
- Query considerations
- Development/test/production separation

Use migration tooling such as Alembic if selected.

Do not manually modify production schemas as the normal workflow.

Do not add indexes without understanding the queries they support.

---

9. API Requirements

The API must be coherent and driven by actual product requirements.

Use:

- Appropriate HTTP methods
- Meaningful HTTP status codes
- Request validation
- Response schemas
- Structured errors
- Pagination where justified
- Filtering/search where justified
- Authentication/authorization where required
- Consistent naming

Do not create endpoints merely to increase endpoint count.

---

10. Validation

Validate all externally supplied input:

- JSON bodies
- Query parameters
- Path parameters
- Authentication-related input
- External service data

Frontend validation improves UX but is not a security boundary.

Backend validation is mandatory.

Invalid requests should return predictable, documented errors.

---

11. Error Handling

Use centralized, consistent error handling.

Production responses must not expose:

- Stack traces
- Database credentials
- API secrets
- Internal infrastructure details
- Sensitive implementation information

Errors should be:

- Safe for users
- Useful to developers
- Consistent
- Logged appropriately
- Documented where relevant

---

12. Configuration and Secrets

Never hardcode credentials.

Use environment-specific configuration for things such as:

```
DATABASE_URL
ENVIRONMENT
SENTRY_DSN
API_KEYS
AZURE_CONFIGURATION
```

The exact variables depend on the final architecture.

Provide:

```
.env.example
```

Never commit real:

```
.env
```

files or credentials.

---

13. Frontend Architecture

Expected structure:

```
Next.js 15
└── App Router
    ├── Server Components where appropriate
    ├── Client Components where interaction requires them
    ├── TypeScript
    ├── Tailwind CSS v4
    ├── shadcn/ui
    ├── Radix UI
    ├── Framer Motion
    └── Sonner
```

Deliberately decide:

- What should be server-rendered?
- What requires client-side interaction?
- Where data fetching should occur
- How loading/error states work
- Which components should be reusable
- Which interactions benefit from animation

Do not use client-side rendering everywhere by default.

---

14. Testing Strategy

Testing is a central project requirement.

The goal is meaningful verification, not vanity coverage.

Backend unit tests

Use pytest or another deliberately selected framework for:

- Validation
- Transformations
- Calculations
- Domain rules
- Utility behavior

Backend integration tests

Test important interactions such as:

```
FastAPI → Database → PostgreSQL
```

Cover:

- Persistence
- Important queries
- Transactions where relevant
- API/database interaction

API tests

Cover:

- Valid requests
- Invalid requests
- Authentication where applicable
- Authorization where applicable
- Correct status codes
- Error contracts
- Edge cases

Frontend tests

Test important user behavior and component behavior where appropriate.

Avoid excessive implementation-detail testing.

End-to-end tests

Use selectively for critical user workflows.

Do not make every small interaction an E2E test.

---

15. Test Philosophy

Every important requirement should have a verification method:

```
Requirement
     ↓
Implementation
     ↓
Test
     ↓
CI
```

A feature is not complete merely because the code exists.

---

16. Docker

Containerize the application.

The final system may include:

```
Next.js application
FastAPI application
PostgreSQL
```

depending on the final deployment architecture.

Containers must:

- Build predictably
- Start predictably
- Receive configuration through environment variables
- Never contain secrets
- Work in CI/CD

Docker Compose may be used for local development if it provides meaningful value.

Do not add Docker Compose simply because it is common.

---

17. GitHub Actions CI

Baseline CI:

```
Push / Pull Request
        ↓
Install dependencies
        ↓
Lint
        ↓
Typecheck
        ↓
Tests
        ↓
Build
        ↓
PASS / FAIL
```

Frontend checks should cover the applicable:

- Lint
- Typecheck
- Tests
- Build

Backend checks should cover the applicable:

- Lint
- Type checking
- Tests

A failing quality gate must prevent unsafe progression.

---

18. Continuous Deployment

Target workflow:

```
Feature branch
      ↓
Pull Request
      ↓
CI
      ↓
Review
      ↓
Merge to main
      ↓
CD
      ↓
Azure
```

Do not deploy unverified code.

The exact deployment workflow should be designed after selecting the Azure architecture.

---

19. Infrastructure-as-Code

Azure infrastructure must eventually be defined in version-controlled files.

Bicep is the preferred IaC option.

Terraform is acceptable only if there is a deliberate reason to use it.

IaC should demonstrate:

- Version-controlled infrastructure
- Repeatable provisioning
- Environment configuration
- Clear dependencies
- Safe infrastructure changes

The environment should not depend on remembering which Azure Portal buttons were clicked.

---

20. Azure Deployment

The application should eventually be publicly accessible over HTTPS.

Potential services include:

- Azure App Service
- Azure Container Apps
- An appropriate Azure PostgreSQL offering

The final architecture must be selected based on:

- Cost
- Simplicity
- Reliability
- Deployment workflow
- Portfolio value
- Operational requirements

Do not provision unnecessary Azure services.

---

21. Observability

Observability is a major project objective.

The system should make it possible to determine:

- Whether the system is healthy
- Whether requests are failing
- Where failures occur
- When failures began
- What errors users encounter

At minimum:

- Structured logs
- Health endpoint
- Error monitoring
- Useful request/error context

---

22. Health Checks

Provide an endpoint such as:

```
GET /health
```

It should be lightweight.

If readiness and liveness need to be separated, design them deliberately.

Do not make health checks perform unnecessarily expensive operations.

---

23. Logging

Use structured logs where practical.

Useful non-sensitive context may include:

- Timestamp
- Severity
- Request/correlation ID
- HTTP method
- Endpoint
- Status code
- Duration
- Relevant application context

Never log:

- Passwords
- Access tokens
- API secrets
- Database credentials
- Unnecessary sensitive information

---

24. Error Monitoring

Use Sentry or another suitable free-tier error-monitoring service if appropriate.

It should help answer:

- What broke?
- When?

- Where?
- How frequently?
- What operation/request was involved?

Never send secrets or inappropriate sensitive data to monitoring.

---

25. API Documentation

Use FastAPI's OpenAPI support as part of the API-development process.

Documentation should include:

- Endpoints
- Methods
- Parameters
- Request bodies
- Response bodies
- Status codes
- Validation errors
- Authentication requirements
- Examples

Documentation must remain synchronized with the implementation.

---

26. Architecture Decision Records

Use ADRs for meaningful architectural decisions.

Possible ADRs:

```
ADR-001 — Why Next.js 15 + App Router
ADR-002 — Why FastAPI
ADR-003 — Why PostgreSQL
ADR-004 — Why a Modular Monolith
ADR-005 — Why Bicep
ADR-006 — Azure deployment architecture
ADR-007 — Testing strategy
ADR-008 — Observability strategy
```

```
ADR-009 — Why Redis is not required
ADR-010 — Why microservices are not required
```

Do not create ADRs for trivial decisions.

---

27. Documentation Structure

Possible structure:

```
docs/
├── CLOUDBENCH_MASTER_SPEC.md
├── ARCHITECTURE.md
├── API.md
├── DATABASE.md
├── FRONTEND.md
├── BACKEND.md
├── TESTING.md
├── SECURITY.md
├── DEPLOYMENT.md
├── OBSERVABILITY.md
├── BUILD_LOG.md
└── adr/
```

The structure may evolve.

---

28. Proposed Repository Structure

A reasonable starting direction:

```
cloudbench/
├── .github/
│   └── workflows/
│
├── apps/
│   ├── web/
│   │   ├── app/
│   │   ├── components/
│   │   ├── lib/
```

```
│   │   └── ...
│   │
│   └── api/
│       ├── app/
│       ├── tests/
│       └── ...
│
├── docs/
│   ├── CLOUDBENCH_MASTER_SPEC.md
│   ├── ARCHITECTURE.md
│   ├── API.md
│   ├── DATABASE.md
│   ├── FRONTEND.md
│   ├── BACKEND.md
│   ├── TESTING.md
│   ├── SECURITY.md
│   ├── DEPLOYMENT.md
│   ├── OBSERVABILITY.md
│   ├── BUILD_LOG.md
│   └── adr/
│
├── infrastructure/
│   └── azure/
│
├── docker/
│
├── CloudBench.pdf
├── README.md
├── .gitignore
└── ...
```

This is a proposed direction, not a rigid structure.

Copilot must inspect the repository and refine it before creating everything.

---

29. Environment Strategy

Reason about at least:

```
development
test
```

```
production
```

Do not mix environments.

Production credentials must never accidentally be used in tests.

Document how each environment receives configuration.

---

30. Security Requirements

At minimum:

- Validate all external input
- Protect secrets
- Use HTTPS in production
- Configure CORS deliberately
- Use secure headers where appropriate
- Prevent internal error leakage
- Use least-privilege credentials where practical
- Keep dependencies updated
- Perform dependency/security checks
- Never commit secrets
- Apply appropriate authentication/authorization where required

Security decisions should be documented.

---

31. Portfolio Presentation

The README should emphasize engineering practices before feature count.

Show:

- Live application
- Architecture diagram
- CI/CD pipeline
- Test status/coverage where meaningful
- Infrastructure-as-Code
- Azure deployment
- Observability

- API documentation
- Important engineering decisions
- Useful screenshots

The portfolio message should be:

> **I can engineer and operate a real software system.**

---

32. Resume Direction

A possible resume statement:

> Built and deployed a production-grade full-stack platform using Next.js, React, TypeScript, FastAPI, PostgreSQL, Docker, GitHub Actions, Infrastructure-as-Code, Azure, automated testing, and error monitoring — engineered for operational readiness rather than feature completeness alone.

The final resume statement must only claim technologies and capabilities that were actually implemented.

---

33. Interview Topics

Be prepared to explain:

Frontend

- Why Next.js?
- Why App Router?
- Server Components vs Client Components
- Why TypeScript?
- Why Tailwind?
- Why shadcn/ui?
- When should Framer Motion be used?
- How is frontend state handled?

Backend

- Why FastAPI?
- API design
- Request validation
- Pydantic models
- Error handling
- Authentication
- Dependency injection
- Business-logic separation

Database

- Why PostgreSQL?
- Indexes
- Constraints
- Transactions
- Migrations
- Query performance
- Data modeling

Testing

- Unit vs integration
- API testing
- Frontend testing
- End-to-end testing
- Mocking
- Test isolation

CI/CD

- Why CI?
- Why test before deployment?
- What happens when CI fails?
- Why deploy after merge?

Infrastructure

- Why IaC?
- Why Bicep?

- How is infrastructure reproduced?
- What happens when infrastructure changes?

Observability

- Logs vs metrics vs error monitoring
- Why error monitoring?
- What should an error contain?
- How would you debug a production failure?

Deployment

- Containers
- Environment configuration
- Health checks
- Rollback considerations
- Azure architecture

Architecture

- Why a modular monolith?
- Why not microservices?
- When would Redis become justified?
- When would queues become justified?
- What would change at 10× scale?

---

34. Scaling Philosophy

CloudBench is not primarily a "millions of requests per second" project.

The primary scalability concept is **operational scalability**:

> Can a development team safely change, test, deploy, monitor, and recover this system?

A system can fail organizationally before it fails technically through:

- Unsafe deployments
- Infrastructure drift
- Poor incident visibility

- Lack of automated testing
- Fear of changing production
- Poor documentation

CloudBench should demonstrate systematic solutions to these problems.

---

35. Scaling Evolution

Stage 1 — MVP

```
Next.js
   ↓
FastAPI
   ↓
PostgreSQL
```

with:

- Tests
- CI
- Docker
- IaC
- Azure deployment
- Monitoring
- Documentation

Stage 2 — Higher traffic

Only when justified:

- Autoscaling
- Caching
- Database optimization
- Read replicas
- Background processing

Stage 3 — Large scale

Potential future additions:

- Advanced autoscaling
- Distributed caching
- Queues
- Partitioning
- Independent services

These are **not MVP requirements**.

---

36. AI-Assisted Development Model

GitHub Copilot is the implementation agent.

The human developer remains the project director.

Workflow:

```
Human requirement
        ↓
Copilot analysis
        ↓
Copilot implementation
        ↓
Automated verification
        ↓
Human review
        ↓
Git checkpoint
        ↓
Next instruction
```

Copilot must not independently decide the entire project scope.

---

37. Copilot Operating Rules

Rule 1 — Inspect Before Modifying

Before meaningful changes:

- Inspect repository structure
- Inspect relevant files
- Inspect tests
- Inspect configuration
- Inspect documentation
- Inspect Git status

Never assume the repository is unchanged.

Rule 2 — Preserve Working Code

Do not blindly overwrite working functionality.

Prefer incremental changes.

Rule 3 — Follow Existing Architecture

Do not introduce frameworks, packages, databases, or patterns without justification.

Rule 4 — Verify

After implementation:

- Run tests
- Run lint
- Run type checks where applicable
- Run builds
- Inspect failures
- Fix failures

Rule 5 — Update Documentation

When architecture, API, database, security, deployment, or configuration changes, update relevant documentation.

Rule 6 — Stop at the Requested Milestone

Do not silently continue into major unrelated work.

38. Copilot Must Not

Copilot must not:

- Build the entire project in one enormous operation
- Invent unrelated features
- Over-engineer
- Add fashionable infrastructure without justification
- Hardcode secrets
- Commit real `.env` secrets
- Skip tests
- Skip verification
- Weaken tests merely to make CI pass
- Fabricate test results
- Fabricate deployment results
- Claim a service is deployed when it is not
- Silently remove working functionality
- Force-push
- Rewrite Git history without explicit instruction
- Delete remote branches without explicit instruction
- Deploy unverified code
- Hide failures

---

39. Git Workflow — MANDATORY

Core Requirement

Git commits and pushes must happen after **meaningful minor achievements/milestones**, not merely after entire phases.

Intended pattern:

```
Minor achievement
      ↓
Verify
      ↓
git status
```

```
      ↓
git diff
      ↓
git add
      ↓
git commit
      ↓
git push
      ↓
Continue
```

Do not wait for an entire phase to finish.

Do not commit every tiny change.

---

40. Commit Frequency

A phase may contain many meaningful achievements.

Example:

```
Phase 1
├── repository foundation → commit + push
├── frontend foundation → commit + push
├── backend foundation → commit + push
├── database connected → commit + push
├── first migration → commit + push
├── first API endpoint → commit + push
├── validation → commit + push
└── phase complete
```

The objective is:

> **Frequent meaningful checkpoints, not one commit per line.**

---

41. Commit-Worthy Achievements

Examples:

- Repository foundation established
- Next.js application bootstrapped
- FastAPI application boots
- PostgreSQL connection established
- Database migration works
- First API endpoint works
- Validation works
- Authentication works
- Important feature works
- Frontend consumes API
- Meaningful test suite established
- Docker setup works
- CI pipeline passes
- OpenAPI documentation integrated
- Azure infrastructure provisioned
- Azure deployment works
- Error monitoring works
- Important documentation milestone completed

---

42. What Does Not Need Its Own Commit?

Do not commit every tiny action.

Examples:

- Renaming one variable
- Adding one import
- Fixing a typo
- Moving a temporary file
- Changing one line with no independent milestone value

---

43. Commit Decision Rule

Before moving to the next substantial task, Copilot should ask internally:

> Have I reached a stable, meaningful achievement that should be preserved independently?

If yes:

```
git status
git diff
git add <relevant-files>
git commit -m "..."
git push
```

If no, continue implementation.

---

44. Commit Message Standard

Prefer descriptive conventional-style messages:

```
feat: add initial health check endpoint
feat: implement user authentication
test: add integration tests for user API
ci: add GitHub Actions test pipeline
infra: provision Azure infrastructure with Bicep
docs: document API authentication flow
```

Avoid:

```
changes
update
stuff
work
fixes
```

---

45. Git Safety

Before committing:

1. Run `git status`
2. Inspect the relevant diff
3. Check for secrets
4. Check for generated junk
5. Run relevant tests/checks

6. Ensure the repository is coherent

Before pushing:

- Verify the commit
- Verify the branch
- Verify the remote
- Verify the push succeeded

Never force-push unless explicitly instructed.

---

46. Milestone Workflow

Every meaningful milestone follows:

```
1. Read requirement
2. Inspect repository
3. Plan
4. Implement
5. Test
6. Fix failures
7. Review diff
8. Update documentation if necessary
9. git add
10. git commit
11. git push
12. Report milestone
13. Stop and wait
```

---

47. Copilot Milestone Report

After each milestone, Copilot should report:

```
MILESTONE COMPLETE

Achievement:
<what was completed>

Files changed:
```

```
<important files>

Tests:
<what was run and result>

Build:
<result>

Lint/typecheck:
<result>

Git:
<commit hash>
<commit message>
<push result>

Known issues:
<any>

Remaining:
<what is intentionally not complete>

Recommended next milestone:
<next logical step>
```

---

48. Optional Build Log

Maintain:

```
docs/BUILD_LOG.md
```

Each entry may contain:

- Date
- Milestone
- Changes
- Tests
- Commit
- Known issues
- Next step

The build log should provide traceability without becoming bureaucracy.

---

49. Development Phases

The initial roadmap should be refined after repository and requirements analysis.

Phase 1 — Product and Core Application

Focus on:

- Select the small product/domain
- Establish architecture
- Design PostgreSQL schema
- Build FastAPI foundation
- Build Next.js foundation
- Establish frontend/API integration
- Implement the core product workflow

Commit after meaningful minor achievements throughout.

Phase 2 — Testing + Quality

Focus on:

- Backend unit tests
- Backend integration tests
- API tests
- Frontend tests
- Critical E2E workflows
- Linting
- Type checking
- Meaningful coverage

Phase 3 — CI/CD + Containers

Focus on:

- Docker
- Local reproducibility

- GitHub Actions
- Automated testing
- Automated builds
- Deployment workflow

Phase 4 — Infrastructure + Azure

Focus on:

- Bicep
- Azure resources
- PostgreSQL infrastructure
- Application deployment
- Environment configuration
- HTTPS
- Repeatable infrastructure

Phase 5 — Observability + Production Readiness

Focus on:

- Health checks
- Structured logs
- Error monitoring
- Production configuration
- Security review
- Failure testing
- Deployment verification

Phase 6 — Documentation + Portfolio Polish

Focus on:

- README
- Architecture documentation
- API documentation
- Database documentation
- Security documentation
- Deployment documentation
- ADRs
- Build log

- Architecture diagram
- Final production review

These phases are guidelines. Do not delay meaningful Git checkpoints until an entire phase is finished.

---

50. MVP Definition

The CloudBench MVP is:

> A publicly deployed, working full-stack application with a Next.js 15 frontend using React 19, TypeScript, Tailwind CSS v4, shadcn/ui/Radix UI, Framer Motion, and Sonner; a Python/FastAPI backend; PostgreSQL persistence; meaningful automated tests; Docker support; a green GitHub Actions CI pipeline; Infrastructure-as-Code; Azure deployment; API documentation; and basic error monitoring.

The product itself does not need to be complex.

The engineering rigor is the point.

---

51. Production Readiness Checklist

Application

- [ ] Product/domain clearly defined
- [ ] Next.js frontend works
- [ ] FastAPI backend works
- [ ] PostgreSQL persistence works
- [ ] Core workflow works
- [ ] Environment configuration works
- [ ] Errors handled consistently

Frontend

- [ ] Next.js 15
- [ ] App Router
- [ ] React 19
- [ ] TypeScript

- [ ] Tailwind CSS v4
- [ ] shadcn/ui
- [ ] Radix UI where appropriate
- [ ] Framer Motion where useful
- [ ] Sonner where appropriate
- [ ] Responsive UI
- [ ] Accessible UI
- [ ] Loading/error/empty states

Backend

- [ ] Python
- [ ] FastAPI
- [ ] Request validation
- [ ] Response schemas
- [ ] Database integration
- [ ] Error handling
- [ ] Logging
- [ ] API documentation
- [ ] Authentication/authorization if required

Testing

- [ ] Backend unit tests
- [ ] Backend integration tests
- [ ] API tests
- [ ] Frontend tests where appropriate
- [ ] Critical E2E workflows
- [ ] CI verifies tests

DevOps

- [ ] Docker
- [ ] GitHub Actions
- [ ] CI
- [ ] CD
- [ ] Azure deployment
- [ ] Bicep/Terraform
- [ ] Environment configuration

Operations

- [ ] Health check
- [ ] Structured logs
- [ ] Error monitoring
- [ ] Production configuration
- [ ] Failure visibility

Security

- [ ] No secrets in Git
- [ ] Input validation
- [ ] HTTPS
- [ ] Deliberate CORS
- [ ] Secure headers where appropriate
- [ ] Production errors do not leak internals
- [ ] Least-privilege credentials where practical
- [ ] Dependency/security checks

Documentation

- [ ] README
- [ ] OpenAPI documentation
- [ ] Architecture documentation
- [ ] Database documentation
- [ ] Frontend documentation
- [ ] Backend documentation
- [ ] Deployment documentation
- [ ] Security documentation
- [ ] ADRs where appropriate
- [ ] Build log where useful

Git

- [ ] Frequent meaningful commits
- [ ] Frequent pushes
- [ ] No secrets in Git
- [ ] Clean history
- [ ] No unnecessary force-pushes

52. Final Project Goal

> **Build and publicly deploy a modest but genuinely useful full-stack software product using Next.js 15, React 19, TypeScript, Tailwind CSS v4, Framer Motion, shadcn/ui, Radix UI, Sonner, Python, FastAPI, and PostgreSQL — while demonstrating professional software-engineering discipline through clean architecture, meaningful testing, Docker, automated CI/CD, Infrastructure-as-Code, Azure deployment, observability, security, documentation, and a disciplined Git history.**

GitHub Copilot acts as the implementation agent under a human-directed, milestone-by-milestone workflow.

The human developer must understand the architecture, implementation, tradeoffs, and engineering decisions rather than merely accepting generated code.